

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №5 по курсу «Дискретный анализ»

Студент: Д. О. Кочев
Преподаватель: А. А. Кухтичев
Группа: М8О-306Б-22
Дата:
Оценка:
Подпись:

Москва, 2024

Лабораторная работа №5

Задача: Найти в заранее известном тексте поступающие на вход образцы с использованием суффиксного массива.

Можно делать как через суффиксное дерево, так и через сортировку.

1 Описание

В задаче требуется реализовать поиск подстроки в строке с использованием суффиксного массива. Суффиксный массив можно получить с помощью суффиксного дерева или через сортировки. Я выбрал второй вариант, он мне показался более изящным и простым.

Для второй реализации требуется использовать стабильную сортировку, а так же дополнительный массив классов эквивалентностей.

После создания суффиксного массива, нам нужно написать алгоритм поиска всех вхождений паттерна в текст. Для этого я буду использовать левосторонний и правосторонний бинарные поиски. Ответом будет любое число между самым левым и самым правым вхождением, включая их.

Пользуясь теоремой из [1]: «При использовании двоичного поиска в массиве P os все вхождения P в T могут быть найдены за время $\Theta(n \log m)$ », мы убеждаемся в скорости и эффективности данного решения.

2 Исходный код

Мой код разделен на две части. Сначала я получаю суффиксный массив через сортировки. Для этого я использую удобную мне структуру *Triple*.

```
1 struct Triple {  
2     int str;  
3     int index;  
4     int eq;  
5 };
```

После увеличения длины текста до степени 2-ки символом \$, переноса в вектор структур *Triple* и заполнения классов эквивалентностей, я запускаю цикл *while*, в котором выполняю алгоритм: вычитание из индексов нужного числа, расстановка классов эквивалентностей относительно индексов, сбалансированная сортировка по классам, обновление классов.

```
1     int count_symbol = 1;  
2     while (big_arr.back().eq != len_text){  
3         for (int i = 0; i < len_text; i++){  
4             big_arr[i].index -= count_symbol;  
5             if (big_arr[i].index < 0)  
6                 big_arr[i].index += len_text;  
7             big_arr[i].eq = dop_eq[big_arr[i].index];  
8         }  
9         std::stable_sort(big_arr.begin(), big_arr.end(), compareByEq);  
10        std::vector<std::pair<int, int>> pair_eq;  
11        for (int i = 0; i < len_text; i++){  
12            int ind = big_arr[i].index + count_symbol;  
13            if (ind >= len_text)  
14                ind -= len_text;  
15            pair_eq.push_back(std::make_pair(big_arr[i].eq, dop_eq[ind]));  
16        }  
17        big_arr[0].eq = 1;  
18        dop_eq[big_arr[0].index] = 1;  
19        for (int i = 1; i < len_text; i++){  
20            if (pair_eq[i] != pair_eq[i - 1]){  
21                big_arr[i].eq = big_arr[i - 1].eq + 1;  
22            }  
23            else big_arr[i].eq = big_arr[i - 1].eq;  
24            dop_eq[big_arr[i].index] = big_arr[i].eq;  
25        }  
26        count_symbol *= 2;  
27    }  
28    std::vector<int> suff_array;  
29    for (int i = count_sentinels; i < len_text; i++){  
30        suff_array.push_back(big_arr[i].index);  
31    }
```

Во второй части кода я реазую поэтапное чтение паттернов, а затем для каждого паттерна использую левосторонний бинарный поиск и правосторонний бинарный поиск, чтобы найти левые и правые границы вхождения строки. Ответом будет все числа в суффиксном массиве между левым и правым индексом, которые я определил, включая их самих.

```

1 | int L = 0;
2 | int R = global_len_text - 1;
3 | int M = (L + R) / 2;
4 | while (R >= L){
5 |     M = (L + R) / 2;
6 |     for (int i = 0; i <= pattern.size(); i++){
7 |         if (i == pattern.size()){
8 |             index_lower = M;
9 |             R = M - 1;
10 |             break;
11 |         }
12 |         if (suff_array[M] + i == global_len_text){
13 |             L = M + 1;
14 |             break;
15 |         }
16 |         if (pattern[i] == text[suff_array[M] + i]){
17 |             continue;
18 |         }
19 |         else if (pattern[i] > text[suff_array[M] + i]){
20 |             L = M + 1;
21 |             break;
22 |         }
23 |         else {
24 |             R = M - 1;
25 |             break;
26 |         }
27 |     }
28 | }

```

laba5.cpp	
bool compareByStr(const Triple &a, const Triple &b)	Функция для стабильной сортировки по символам
bool compareByEq(const Triple &a, const Triple &b)	Функция для стабильной сортировки по классам эквивалентности

3 Консоль

```
dmitrykochev$ cat test1.txt
abcdabc
abcd
bcd
bc
ab
q
c
dmitrykochev$ ./laba5 <test1.txt
1: 1
2: 2
3: 2,6
4: 1,5
6: 3,7
dmitrykochev$ cat test2.txt
abacaba
a
aba
caba
c
ba
dmitrykochev$ ./laba5 <test2.txt
1: 1,3,5,7
2: 1,5
3: 4
4: 4
5: 2,6
```

4 Тест производительности

Тест производительности представляет из себя следующее: одни и те же паттерны и текст мы будем использовать как в алгоритме суффиксного массива, так и в наивном алгоритме поиска подстроки в строке. Тестов будет три, тест характеризуется двумя числами: количество слов в тексте и количество паттернов. Паттерны будут специально генерироваться короткими, в несколько символов, и длинными. Первый тест будет содержать 10 паттернов, 100 символов в тексте. Второй 10^3 паттернов, 10^4 символов в тексте. Третий 10^4 паттернов, 10^5 символов.

```
dmitrykochev$ ./benchmark <test1.txt
Suffix array algorithm time: 202us
Native algorithm time: 375us
dmitrykochev$ ./benchmark <test2.txt
Suffix array algorithm time: 116483us
Native algorithm time: 156583us
dmitrykochev$ ./benchmark <test3.txt
Suffix array algorithm time: 9980583us
Native algorithm time: 11092271us
```

Как можно заметить, при увеличении текста алгоритм с использованием суффиксного массива становится заметно быстрее наивного алгоритма.

Действительно, как говорилось выше, создание суффиксного массива по сложности составляет $\Theta(n \log n)$, а поиск паттерна занимает $\Theta(m \log n)$, в то же время в наивном алгоритме поиск происходит за $\Theta(m * n)$, где m - длина текста, n - длина паттерна. Мы видим, что уже на маленьких тестах суффиксный массив работает быстрее наивного алгоритма.

5 Выводы

Выполнив пятую лабораторную работу по курсу «Дискретный анализ», я научился искать подстроку в суффиксном массиве, а так же создавать его без суффиксного дерева. Безусловно, это дало мне ценный опыт в написании алгоритмов поиска подстроки своими руками с нуля. Ввиду особенностей данного алгоритма, мне также приходилось использовать нестандартные решения хранения данных. Во время выполнения я сталкивался с большим количеством проблем, так как алгоритм довольно сложен для осознания. В итоге я был удивлен, насколько создание суффиксного массива через сортировки - изящное решение. Все это в целом очень важный опыт и знания. Они очень помогут мне в будущей разработке сложных задач и программ.

Список литературы

- [1] Dan Gusfield, *Algorithms on string, trees, and sequences*, Cambridge University, 1997.